

INTERNAL BRIEFING · PRESENTATION PREP

Migrating JobMgr & WeighApp from SCS to ROS2/DDS

What the project is, how the approach was chosen, what got built, what broke and got fixed, and a ready set of answers for the questions this tends to provoke.

CONTENTS

- | | |
|--|--------------------------------|
| 1. The one-paragraph version | 5. How messages actually flow |
| 2. What existed before | 6. Why one thread, not two |
| 3. The core decision: communication-only | 7. The verification discipline |
| 4. The new architecture | 8. Current state |
| | 9. Anticipated questions |

01 The one-paragraph version

LpsSaJobMgrApp ("JobMgr") and **LpsSaWeighApp** ("WeighApp") are two real, already-running industrial apps — part of a machine load-weighing system — built on Caterpillar's internal AIS task framework. They talk to each other and to a handful of other components (a legacy UI, hardware I/O, a display app) over a proprietary, shared-memory-based transport called **SCS**. The project: replace that transport with **ROS2/DDS**, channel by channel, without touching anything else — not the business logic, not the real-time scheduling, not the NVM persistence, not the watchdog. Everything the app *does* stays identical; only how it *talks* changes.

02 What existed before

Every SCS channel in the old system followed the same shape: a macro (`DefineInputChannel` / `DefineOutputChannel`) generated a concrete class per channel, which multiply-inherited from an abstract `InputInterface<T>` / `OutputInterface<T>` and a lower-level `ChannelInterface<T>` that did the real shared-memory read/write. The app itself never saw the concrete type — it held a pointer typed as the *abstract* base, and a string-keyed registry called `task->TaskFaceDb` handed back the real instance at startup.

`Task::InterfaceDb` handed back the real instance at startup

(`InterfaceDb::fetch("SomeChannelName")`) or (`::bind(name, ptr)`).

Two access patterns existed side by side: a plain **poll** (`while (channel→get(x)) { ... }`), drained once per tick), and a **push callback** (`addNewDataSlot()`, boost::signals2-based, fired from some thread other than the app's own — which is exactly why the classes that used it needed real mutexes and condition variables).

03 The core decision: communication-only, not a rewrite

Two options were on the table early on:

1. **Full rewrite** — turn both apps into native ROS2 nodes, drop the AIS task framework entirely.
2. **Communication-only** — keep everything about how the apps run exactly as-is; swap only the transport underneath each channel.

DECISION

Communication-only. A full rewrite would have meant re-solving, from scratch, things the AIS task framework already gives for free: real-time OEL scheduling, WeighApp's independent background NVM read/write task, watchdog servicing, and process supervision. None of that is what this project was actually trying to change.

One refinement mattered: an early version of the plan converted *only some* channels to DDS, leaving others on raw SCS pointers inside the same app. That was explicitly rejected — "*ROS2 needs to be brought into the picture as a lead actor,*" not a partial patch. The final design: apps stay `task::Task` subclasses (all the scheduling guarantees stay), but **every** channel becomes a DDS wrapper — zero raw SCS objects held by either app once conversion is complete. Whatever still needs to reach an un-migrated component goes through one shared relay process (the **Bridge**), not a wrapper per app.

04 The new architecture

The wrapper layer

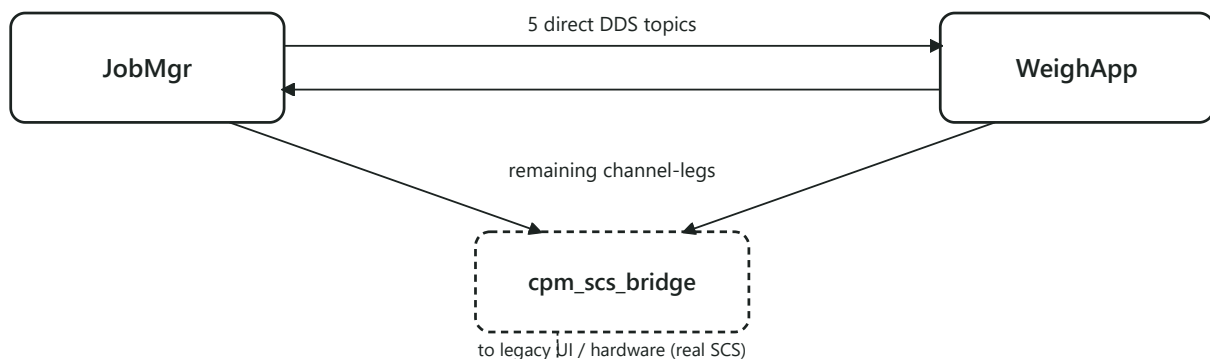
Two small template classes, `RosInputInterface<T>` and `RosOutputInterface<T>`, are the whole translation layer. They were deliberately designed to be near drop-in replacements for the old SCS interface classes — same method names (`get()`, `publish()`), so call sites throughout the existing codebase needed minimal changes. Under the hood, `RosInputInterface<T>` wraps an `rclcpp::Subscription` whose callback pushes every arriving message into a real `std::queue<T>`; `get()` pops one at a time.

Three new message packages

Every real SCS channel got a matching ROS2 `.msg` definition, split across three packages by a strict dependency rule: `cpm_common_interfaces` holds anything that genuinely crosses the JobMgr↔WeighApp boundary (used by both, depends on neither); `job_mgr_interfaces` and `weigh_app_interfaces` hold what's local to each app. Neither app-specific package may depend on the other — only on the shared one.

Direct wiring vs. the Bridge

Five channels flow directly between JobMgr and WeighApp over native DDS topics — no intermediary. Everything else that used to reach a not-yet-migrated peer (legacy UI, hardware switches, a display app) goes through `cpm_scs_bridge`: a new process that binds the *real* SCS side (still using `task::InterfaceDb`, same as the original apps) on one side, and a ROS2 wrapper on the same topic JobMgr/WeighApp already use on the other, relaying between them once per tick.



The 5 shared channels go straight DDS-to-DDS. Everything JobMgr or WeighApp still needs to exchange with an un-migrated component routes through one shared Bridge process.

05 How messages actually flow

This is the one worth having a crisp, confident answer for — it came up once already.

The short version

ROS2/DDS is publish-subscribe. A publisher sends a message onto a named **topic** without knowing or caring who's listening. Any node subscribed to that topic gets its own copy, delivered asynchronously. Our wrapper doesn't invent anything new here — it rides directly on that mechanism.

What happens inside `RosInputInterface<T>`

```

template<typename T>
class RosInputInterface {

```

```

public:
    RosInputInterface(const rclcpp::Node::SharedPtr& node,
                      const std::string& topicName, size_t qosDepth = 10) {
        subscription_ = node->create_subscription<T>(topicName, qosDepth,
            [this](const typename T::SharedPtr msg) { queue_.push(*msg); });
    }
    bool get(T& data) {
        if (queue_.empty()) return false;
        data = queue_.front(); queue_.pop(); return true;
    }
private:
    std::queue<T> queue_;
    ...
};

```

When a message arrives, ROS2 invokes that subscription callback — which does exactly one thing: push it onto a plain `std::queue<T>`. It doesn't process it, doesn't act on it, just stores it. Nothing else happens until the app explicitly asks.

Draining it

Every task's `executive()` runs on a fixed cycle. Once per cycle, it calls `executor_.spin_some()` — that's the point where every pending callback across every subscribed topic actually fires, filling whatever queues have new data. Immediately after, the app's own code drains each queue it cares about with a `while (get()) { ... }` loop — read the analogy as a mailbox: messages pile up between checks, and every check empties the box completely, in arrival order, before moving on. If two messages arrived between one tick and the next, both get processed — nothing is silently overwritten or dropped.

WHY A QUEUE, NOT A SINGLE CACHED VALUE

An earlier reference implementation cached only the latest message (overwrite-on-arrival). Real SCS `get()` is called in drain loops throughout this codebase specifically because more than one update can land between polls — a single cached value would silently lose data. A real `std::queue<T>` was used instead, deliberately.

The full per-tick sequence

```

executive()
├─ executor_.spin_some()           ← every subscription callback fires here,
│                                 queues get filled for this cycle
├─ SomeChannelIn->get(x)           ← drains that channel's queue,
│   while (get(x)) { ... }         oldest message first
├─ ... business logic runs on whatever was just drained ...
└─ SomeChannelOut->publish(y)      ← hands y to rclcpp, which sends it
                                   over DDS to every current subscriber

```

06 Why one thread, not two

This is worth being able to defend on its own, because it's the single biggest architectural bet in the whole design.

`spin_some()` is called synchronously, from inside the app's own `executive()`, on the app's one and only thread. The callback that fills the queue and the code that drains it (`get()`) therefore *never run concurrently* — one always finishes before the other starts. That's what lets the wrapper skip a mutex entirely: there's no race to guard against, by construction, not by careful locking.

The real SCS system didn't have this property — its push-callback path (`addNewDataSlot()`) fired from a separate thread, which is why the original callback-based classes genuinely needed a mutex and a condition variable. A prototype was built exploring what it would take to bring that exact behavior back (a real background thread spinning the executor continuously, restoring real blocking waits like `waitForResponse()`) — it works, but it reintroduces the entire class of concurrency bugs the single-thread design exists to avoid: races, deadlocks, shutdown-ordering edge cases. It stayed a comparison prototype, not the adopted approach.

IF ASKED "ISN'T POLLING WORSE THAN A REAL CALLBACK?"

The honest tradeoff: the poll-based design can't do a true blocking wait — `waitForTxData()` checks once per tick and returns immediately either way, rather than sleeping until data arrives. In practice this doesn't cost much, because the tick loop itself keeps checking every cycle regardless — a request sent this tick is very likely satisfied by next tick's check. It's a real, acknowledged behavioral difference from the original system, deliberately accepted in exchange for a design that cannot deadlock or race.

07 The verification discipline

A recurring problem: several real SCS struct definitions (the ones a channel's message actually carries) live in closed, prebuilt submodules that weren't present in the working checkout. Two ways this got handled, and it matters which:

- **Reconstruct from usage, mark it honestly.** When a struct's real header was missing, the fields actually used were traced through every real call site in the codebase, and only those were captured — with an explicit, loud `UNVERIFIED PLACEHOLDER` comment anywhere a numeric value (like a GPIO pin number) had to stand in for one that genuinely wasn't known. Never a plausible-looking guess presented as fact.
- **Go back and fix it once real source appears.** A newly-provided source directory later resolved several of these placeholders for real (exact GPIO pin numbers, real

enum values). Every one of those was independently re-verified against the actual header before being applied — and the messages themselves were expanded to match the real struct shape where that struct turned out to carry more than the placeholder assumed, not left narrower than necessary just because the narrow version happened to already work.

This same discipline caught real bugs before they'd have mattered: a channel's response object was never actually getting timestamped despite a comment claiming it was (silently defeating a freshness check downstream), and a callback registration was being made against a method that didn't exist on the new wrapper type at all — a bug that would have compiled clean on a wrapper with a slightly different shape and only failed silently at runtime. Both were found by systematically re-checking every call site against the real class being called, not by trusting an existing comment.

08 Current state

PIECE	STATUS
Interface message definitions (all channels, both apps)	Done
ros2_wrapper (RosInputInterface / RosOutputInterface)	Done
JobMgr — every channel converted to DDS	Done
WeighApp — every channel converted to DDS	Done
Direct DDS wiring, JobMgr↔WeighApp (5 shared legs)	Done, verified
Stage 1 minimal rollout slice (3 self-contained channels)	Done, bug-hunted
cpm_scs_bridge — JobMgr-side relays	5 of 13 legs implemented; rest scoped, not guessed
cpm_scs_bridge — WeighApp-side relays	Not started
Real compiler/build verification	Not yet possible in this environment — no build toolchain available

BE UPFRONT ABOUT THIS ONE

Every piece of this has been verified statically — real header cross-referencing, real call-site tracing, manual type-rule checking — but none of it has gone through an actual compiler yet. That's an honest, explicitly tracked gap, not an

oversight.

09 Anticipated questions

Q Why not just rewrite both apps as native ROS2 nodes?

A Because the AIS task framework already provides real-time scheduling, NVM persistence, watchdog servicing, and process supervision for free — none of which is what this project set out to change. Rewriting would mean re-solving all of that from scratch for no functional gain.

Q How do you know a message isn't dropped between checks?

A Because the wrapper is backed by a real queue, not a single cached value, and every drain is a `while` loop that empties it completely before moving on — arrival order preserved, nothing overwritten.

Q Is this thread-safe?

A Yes, and for a specific reason: the callback that fills the queue and the code that drains it run on the same single thread, never concurrently, so there's no race to guard against in the first place — no mutex needed, by construction.

Q What happens if you send a request and the response is slow?

A The freshness gate handles it: a request sets a "not valid until at least this timestamp" marker; tx data is only trusted once its own timestamp clears that marker. If the response takes longer than one tick, the check just comes back "not yet" and tries again next tick — no blocking, no deadlock risk.

Q What's not actually done yet?

A The Bridge — the relay process for whatever still needs to reach not-yet-migrated components (legacy UI, some hardware). 5 of JobMgr's 13 legs are implemented; the rest are scoped but intentionally not rushed, especially the ones where the real SCS struct still isn't available anywhere in the checkout. WeighApp's ~19 legs haven't been started.

Q How confident are you this actually works?

A High confidence in correctness of the logic — every conversion was checked field-by-field against real source, and static analysis caught two genuine pre-existing bugs. Zero confidence claimed on "it compiles and runs on target hardware" — that step hasn't happened yet, and that's said plainly, rather than implied.

that's said plainly rather than implied.

Prepared from the working project history — reflects the state of the migration as of